

VI 8 Bits Restoring Divisor

Contreras R Orlando^{1,2*} and Garcia B Iker^{1,2*†}

³Departamento de Sistemas Electrónicos, UAA, Av. Universidad 940,
Aguascalientes, 20131, Aguascalientes, México.

*Corresponding author(s). E-mail(s): [al348390,al307630}@edu.uaa.mx](mailto:{al348390,al307630}@edu.uaa.mx);

†Ambos autores contribuyeron de forma equitativa a este trabajo.

Abstract

El presente documento busca implementar de forma efectiva la división con restauración, dicha implementación será descrita de forma combinacional y con sentencias concurrentes.

Keywords: VHDL, FPGA, Divisor, restauración, numerador, denominador

1 Introducción

Este divisor se destaca por realizar la operación mediante el algoritmo de restauración, y para manejar signos de manera eficiente se usa una Xor. Esta técnica permite manejar correctamente los signos durante la división y sirve como una base fundamental para comprender métodos más avanzados y eficientes de división en sistemas digitales.

El procesamiento eficiente de señales digitales y aplicaciones de alto rendimiento requieren operaciones aritméticas rápidas y eficientes. Entre ellas, la división de números con signo es una de las más importantes, ya que es fundamental en algoritmos de procesamiento de imágenes, inteligencia artificial y sistemas embebidos.

En este contexto, el uso de circuitos diseñados en **FPGA** ofrece ventajas significativas en términos de velocidad, paralelismo y consumo energético en comparación con soluciones basadas en procesadores convencionales, haciendo que cada vez mas los componentes electrónicos se vuelvan mas especializados y mas adaptados al proceso en cuestion.

2 Metodología

2.1 Algoritmo de división

En este caso se utilizó el algoritmo con restauración, el algoritmo consistía en prácticamente concatenarle un '1' hasta la derecha y realizar una resta con el denominador, si el resultado daba un valor negativo, se tomaba el valor antes de restar y se agrega un '0' en un vector, en caso contrario (siendo positivo) se tomaría el valor del resultado y se agregaría un '1' al vector, éste vector sería nuestro resultado. Este algoritmo repetiría por 8 veces, siendo el valor del último caso nuestro resultado.

Este algoritmo puede ser descrito de forma secuencial o de forma combinacional, nosotros decidimos implementarlo de forma combinacional priorizando la velocidad de éste mismo, sin embargo debe considerarse que puede ocasionar errores al momento de hacerlo debido a que se ejecuta en paralelo podría tomar un valor erróneo, esto queda más que nada como advertencia debido a que con nosotros no hubo ningún problema con respecto de eso.

3 Resultados

Con la simulación podemos concluir que la práctica fue exitosa, la división la realiza de forma combinacional por lo que es eficiente y nos arroja el resultado en qout mientras que el residuo se encuentra en gbg.

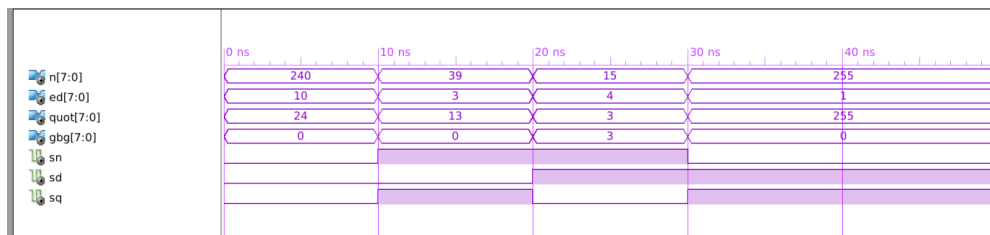


Fig. 1 Testbench realizado por Xilinx

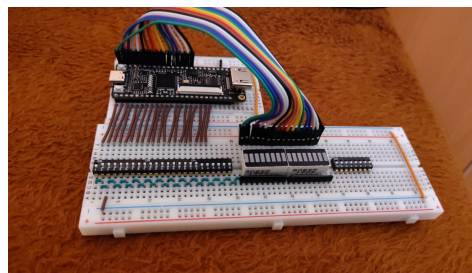


Fig. 2 Circuito Armado

Device Utilization Summary					
Slice Logic Utilization	Used	Available	Utilization	Note(s)	
Number of Slice Registers	0	11,440	0%		
Number of Slice LUTs	187	5,720	3%		
Number used as logic	187	5,720	3%		
Number using O6 output only	165				
Number using O5 output only	0				
Number using O5 and O6	22				
Number used as ROM	0				
Number used as Memory	0	1,440	0%		
Number of occupied Slices	85	1,430	5%		
Number of MUXCYs used	0	2,860	0%		
Number of LUT Flip Flop pairs used	187				
Number with an unused Flip Flop	187	187	100%		
Number with an unused LUT	0	187	0%		
Number of fully used LUT-FF pairs	0	187	0%		
Number of slice register sites lost to control set restrictions	0	11,440	0%		
Number of bonded IOBs	35	102	34%		
Number of RAMB16BWERS	0	32	0%		
Number of RAMB8BWERS	0	64	0%		
Number of BUFIO2/BUFIO2_CLKs	0	32	0%		
Number of BUFIO2FB/BUFIO2FB_2CLKs	0	32	0%		
Number of BUFG/BUFGMUXs	0	16	0%		
Number of DCM/DCM_CLKGENs	0	4	0%		
Number of ILOGIC2/I5SERDES2s	0	200	0%		
Number of IODELAY2/IODRP2/IODRP2_MCBs	0	200	0%		
Number of OLOGIC2/O5SERDES2s	0	200	0%		
Number of BSCANs	0	4	0%		
Number of BUFHs	0	128	0%		
Number of BUFPLLs	0	8	0%		
Number of BUFPLL_MCBs	0	4	0%		
Number of DSP48A1s	0	16	0%		

Fig. 3 Summary report generated by Xilinx

4 Conclusiones

Esta práctica nos resultó menos compleja en comparación con las anteriores, ya que contábamos con una base teórica previa sobre la operación de división gracias a nuestras lecturas del libro de Omondi. Esto nos permitió comprender con mayor rapidez los fundamentos detrás del diseño del divisor y enfocarnos más en su implementación práctica.

Cabe destacar el papel fundamental que jugó la FPGA Tang Nano 9K (fabricada por Sipeed) en el desarrollo de esta práctica. Su facilidad de uso, tamaño compacto y buena documentación técnica permitieron una implementación rápida, pruebas eficientes y una mejor comprensión del comportamiento del circuito en tiempo real.

5 Anexos

Esto es la parte más importante de los RTL, si se gusta observar con mayor detalle, los PDFs estarán en el documento adjunto. Asimismo se podrá observar el código actualizado en: [GitHub Repository](#)

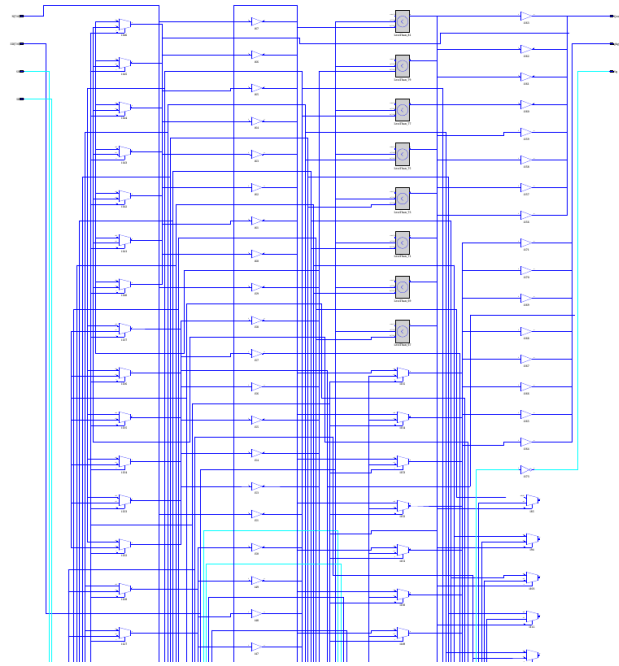


Fig. 4 RTL del Componente principal

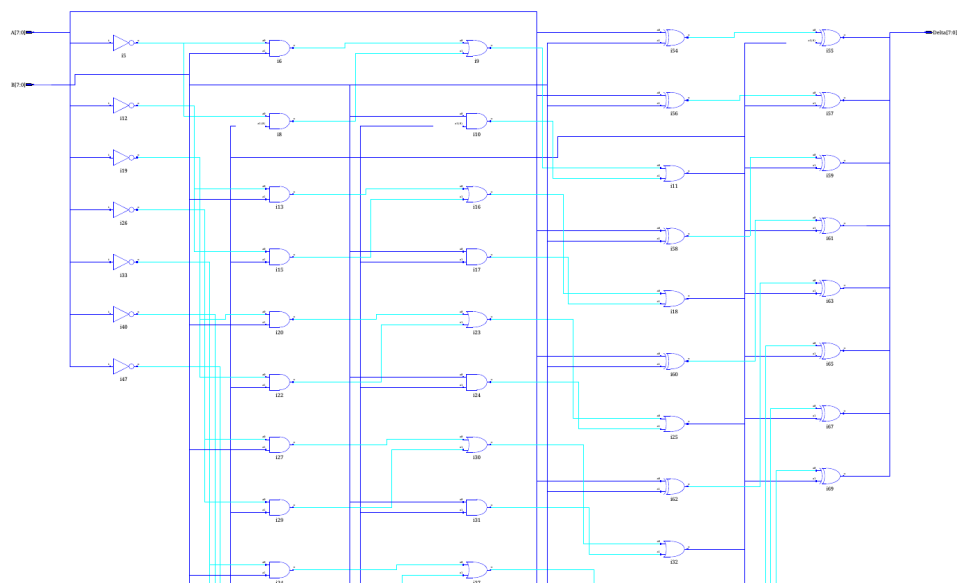


Fig. 5 RTL del Componente Restador